Apprentissage par renforcement multi-robots pour un entrepôt automatisé

Juliette Girardin
Loïc Lamour
Lohann Colle
Antoine de Chabannes

30 octobre 2026

1 Introduction

Dans un entrepôt automatisé, une flotte de chariots autonomes transporte des caisses entre étagères et quais, sur une grille d'allées d'une case de large où circulent aussi des humains. Chaque robot doit trouver son chemin, et la flotte doit se coordonner pour que deux robots ne se bloquent pas face à face. Avec chaque robot ajouté, le nombre de situations possibles est multiplié, ce qui permet de voir concrètement où une méthode tabulaire cède la place à une méthode approchée. Nous ne cherchons pas à apprendre qui fait quoi : une file attribue les tâches aux robots libres, comme le ferait le logiciel de gestion de l'entrepôt, et l'apprentissage porte uniquement sur leurs déplacements.

Nous comparons deux approches. Dans le contrôle centralisé (Lohann Colle et Antoine de Chabannes), un seul agent choisit les actions de toute la flotte : nous montrons qu'une Q-table jointe devient inutilisable dès 3 robots, puis qu'un PPO masqué y remédie et bat une heuristique de plus court chemin. Dans le contrôle décentralisé (Juliette Girardin et Loïc Lamour), chaque robot apprend sa propre politique, au prix d'un environnement non stationnaire [5].

2 L'environnement

L'entrepôt est une grille d'allées, d'étagères et de quais : soit une carte fixe 7x6, soit un labyrinthe aléatoire renouvelé à chaque épisode. Des tâches de prélèvement ou de rangement arrivent en continu et sont attribuées aux robots libres. À chaque pas, chaque robot choisit parmi 6 actions : avancer, reculer, tourner à gauche ou à droite, prendre ou déposer une caisse. Il connaît le plan de l'entrepôt et sa tâche, mais ne voit les autres robots et les humains qu'à travers quatre rayons. Il gagne 10 points par caisse livrée et perd 0,05 par pas, 1 par blocage et 0,1 par action invalide. Le scénario le plus difficile, big, combine un labyrinthe 21x21, 6 robots aux capacités différentes, des tâches prioritaires et 3 humains. Chaque résultat est mesuré sur 100 épisodes dont les graines n'ont jamais servi à l'entraînement : avec les labyrinthes aléatoires, les cartes d'évaluation sont donc inédites.

3 Méthodes

Les deux approches sont comparées à une même référence, un glouton : à chaque pas, chaque robot calcule un plus court chemin vers sa cible en contournant les robots et humains visibles. Une action aléatoire une fois sur dix évite que deux robots face à face se bloquent indéfiniment.

3.1 Approche centralisée

3.1.1 Q-learning tabulaire

Un seul contrôleur choisit l'action jointe de toute la flotte et l'apprend par Q-learning [6]. L'état regroupe la position, l'orientation et la tâche de chaque robot. Sur la carte 7x6, cela fait 2312^n états et 6^n actions jointes pour n robots : 5 millions d'états et 36 actions à 2 robots, 29 000 milliards d'états et 1296 actions à 4 robots. La table est plafonnée à 600 Mo de mémoire.

3.1.2 PPO

Pour dépasser cette limite, un seul réseau de neurones contrôle toute la flotte. Il ne lit pas un état joint : pour chaque robot, il combine ses propres caractéristiques à ses relations avec chacun des autres robots (distance, « l'autre est sur mon chemin », « nous visons la même case »), puis produit son action. Comme ces entrées ne dépendent ni de la carte ni du nombre de robots, le même réseau s'applique à toute taille de flotte et à des labyrinthes jamais vus.

Le réseau a deux sorties : une politique, qui donne la probabilité de chaque action, et un critique, qui estime la valeur de l'état et sert à juger si une action a fait mieux que prévu. PPO [4] met à jour la politique sur des trajectoires récentes en limitant l'ampleur de chaque changement, ce qui stabilise l'apprentissage. Nous l'avons préféré à une méthode à mémoire de rejeu, car les robots apprennent ensemble et le labyrinthe change à chaque épisode : d'anciennes transitions décriraient une dynamique qui n'existe plus. Chaque taille de flotte est entraînée 60 minutes sur 8 entrepôts simulés en parallèle.

Quatre choix, mesurés un par un, ont rendu l'apprentissage possible. Masquer les actions impossibles [2] fait passer les actions invalides d'environ 1128 par épisode à 0. Donner au réseau la distance de plus court chemin à sa cible et la direction à suivre, plutôt que la carte entière, lui évite d'avoir à calculer un chemin qu'il ne sait pas trouver seul. Ajouter un indicateur de voisin immédiat dans chaque direction fait passer les blocages de 45,2 à 12,6 par épisode. Enfin, tirer les actions au hasard selon la politique, plutôt que de prendre la plus probable, empêche deux robots identiques face à face de se refléter indéfiniment, ce qui coûtait 211 blocages par épisode.

3.2 Approche décentralisée

Dans l'approche décentralisée, la décision n'est plus confiée à un contrôleur unique : chaque robot agit comme un agent autonome qui choisit ses propres actions à partir de ses observations locales. Cela transforme le problème en un jeu stochastique partiellement observable (POSG) [10]. La dynamique de l'environnement devient non stationnaire du point de view de chaque robot [5].Nous comparons trois algorithmes d'apprentissage décentralisé à une ligne de base tabulaire.

3.2.1 IQL Tabulaire

L'Independent Q-Learning (IQL) adapte le Q-learning standard en attribuant une Q-table indépendante à chaque robot. Chaque agent traite les autres robots comme des éléments passifs de l'environnement. L'espace d'état local (10^7 configurations possibles par robot) entraîne une saturation rapide de la mémoire et une incapacité à généraliser en dehors des états découverts durant l'exploration.

3.2.2 IDQN (Independent Deep Q-Learning)

Pour surmonter le fléau de la dimensionnalité, IDQN remplace la Q-table par un réseau de neurones profond (DQN) [7] partagé ou instancié par robot. Le réseau prend en entrée un vecteur d'observation local à 11 dimensions :

— La position normalisée du robot (x, y) et son orientation ($heading \in [0, 1]$) ;
— L'état de charge ($carrying$), l'indicateur de tâche (has_task) et le vecteur directionnel relatif (dx, dy) vers la cible actuelle ;

— L'état d'occupation des quatre cases adjacentes (voisins immédiats).

Afin d'éviter que le robot ne pointe vers le quai les mains vides, le vecteur (dx, dy) s'oriente dynamiquement : il calcule la distance vers la caisse lorsque $carrying = 0$, puis bascule vers le quai de déchargement dès que la caisse est récupérée ($carrying = 1$).

Un masque d'actions strict (*Action Masking*) [2] est appliqué pendant l'exploration ϵ-greedy et l'inférence. Il désactive la marche arrière pour éliminer les collisions aveugles et masque à -10^9 dans les logits les actions physiquement impossibles (avancer dans un mur, exécuter un *PICKUP* hors zone ou sans caisse disponible).

3.2.3 IPPO (Independent PPO)

IPPO déploie un algorithme Acteur-Critique décentralisé sur chaque agent, adaptant PPO au cadre multi-agents indépendant [9]. L'Acteur produit une distribution sur les 6 actions à partir de l'observation 11D locale, tandis que le Critique estime la valeur d'état locale $V(s_i)$. L'entraînement s'effectue de manière stochastique via des mises à jour PPO [4]. Le masquage d'actions est appliqué sur les logits d'action de l'Acteur.

3.2.4 MAPPO (Multi-Agent PPO - CTDE)

MAPPO repose sur le paradigme d'entraînement centralisé avec exécution décentralisée (CTDE), dont l'efficacité en environnement coopératif contraint a été démontrée par Yu et al. [8] (*Centralized Training with Decentralized Execution* – CTDE). Lors de l'exécution (inférence sous Isaac Sim), chaque robot utilise uniquement son Acteur décentralisé alimenté par son observation locale 11D. Durant l'entraînement, le Critique est centralisé : il prend en entrée l'état joint concaténé de la flotte (22D pour 2 robots, $V(S_1, S_2)$). Cette valeur globale stabilise le calcul de l'avantage et réduit la variance causée par la non-stationnarité de l'environnement décentralisé.

4 Résultats

4.1 Approche centralisée

Nous comparons la Q-table, le glouton et PPO sur la carte 7x6, sans humains, de 2 à 4 robots. La Q-table est entraînée 90 minutes à 2 robots et 60 minutes au-delà.

Robots	États théoriques	États inconnus	Q-table	Glouton	PPO
2	5 millions	0 %	1,23 ± 2,05	8,57 ± 1,74	12,82 ± 1,03
3	12 milliards	99,5 %	0	10,56 ± 2,56	16,86 ± 1,74
4	29 000 milliards	100 %	0	10,30 ± 3,84	17,97 ± 2,11

TABLE 1 – Livraisons par épisode (moyenne ± écart-type sur 100 épisodes). États inconnus : part des états rencontrés en évaluation que la Q-table n'a jamais vus.

La Q-table ne livre qu'une caisse par épisode à 2 robots, après 57 millions de pas. Dès 3 robots, sa mémoire sature en moins d'un million de pas et elle ne connaît presque aucun des états rencontrés en évaluation.

PPO dépasse le glouton à chaque taille, et l'écart grandit avec la flotte : +50 % à 2 robots, +74 % à 4. Il atteint son plateau en moins d'un million de pas, soit plus de 50 fois moins d'expérience que la Q-table à 2 robots. Le glouton, lui, plafonne dès 3 robots, car il passe son temps à contourner les autres.

Sur le scénario big, avec 100 labyrinthes jamais vus, PPO livre 18,3 caisses par épisode contre 10,6 pour le glouton. Ce modèle a été entraîné 12 heures, avec une version antérieure du réseau sans relations entre paires de robots. Ses trajectoires sont aussi plus directes : le glouton revient sur la case qu'il vient de quitter dans un tiers de ses déplacements, car chaque pas change ce que voient ses rayons et il hésite entre deux chemins, contre 8 % pour PPO.

4.2 Approche décentralisée

Nous évaluons les performances des méthodes décentralisées (IQL, IDQN, IPPO et MAPPO) sur la carte fixe 7×6 avec 2 robots, en les comparant à la référence du glouton déterministe sur 200 pas d'évaluation.

Algorithme	Livraisons	Actions invalides	Pas bloqués	Diagnostic technique
Glouton	8,57 ± 1,74	0	0	Baseline déterministe A^*
IQL Tabulaire	0,00 ± 0,00	202	0	Fléau de la dimensionnalité (2×10^9 états)
IDQN (SKRL)	0,00 ± 0,00	**0**	400	Masque 100 % étanche, politique passive
IPPO (SKRL)	0,00 ± 0,00	196	0	Exploration stochastique en goulot
MAPPO (SKRL)	0,00 ± 0,00	199	0	Critique joint 22D, congestion spatiale

TABLE 2 – Performance des algorithmes décentralisés sur la carte 7×6 (2 robots, 200 pas max).

L'intégration de l'orientation (*heading*) dans le vecteur 11D et du masque d'actions strict transforme radicalement le comportement d'IDQN : le nombre d'actions invalides chute à **0**, confirmant la parfaite étanchéité du masquage applicatif. En inférence déterministe ($\text{argmax}\, Q$), l'agent IDQN adopte une « politique de sécurité passive » : ne parvenant pas à contourner les étagères sans subir de pénalités intermédiaires, il reste stationnaire sur sa case de départ, générant 400 pas bloqués (2 robots $\times$ 200 pas sans progression).

IPPO et MAPPO réduisent le nombre d'actions invalides d'environ 50 % par rapport aux versions non masquées. Toutefois, la nature stochastique des politiques PPO et l'absence de communication explicite entraînent des verrous spatiaux (*deadlocks*) lorsque les deux robots se rencontrent dans l'allée centrale menant au dépôt.

5 Discussion et limites

5.1 Comparaison des deux approches

L'approche centralisée (PPO) résout la coordination en traitant la flotte comme un système unique. Elle atteint d'excellentes performances (jusqu'à 17,97 livraisons à 4 robots) au prix d'une complexité combinatoire (6^n actions jointes) qui rend la méthode tabulaire irréalisable dès 3 robots.

À l'inverse, l'approche décentralisée supprime ce goulot combinatoire en attribuant un réseau léger à chaque robot, permettant un traitement en parallèle. Néanmoins, elle se heurte à deux verrous majeurs :

1. **La non-stationnarité et la coordination locale :** Sans canal de communication ni grille d'occupation globale, les agents décentralisés perçoivent les autres robots comme des obstacles dynamiques imprévisibles, provoquant des blocages réciproques dans les allées étroites.
2. **L'apprentissage de sous-objectifs séquentiels :** Contrairement au PPO centralisé qui bénéficie d'une vision globale du trajet, un agent décentralisé doit apprendre seul la séquence *Naviguer* $\rightarrow$ *Aligner l'orientation* $\rightarrow$ *PICKUP* $\rightarrow$ *Naviguer* $\rightarrow$ *DROP*.

5.2 Approche centralisée

Le contrôle centralisé tabulaire illustre concrètement la malédiction de la dimension [1] : l'espace d'états et d'actions est multiplié à chaque robot ajouté, et la table sature avant même d'avoir pu apprendre. PPO garde l'idée d'un contrôle commun à toute la flotte, mais remplace l'état joint par des caractéristiques propres à chaque robot et à ses relations avec les autres. C'est ce changement de représentation, plus que l'algorithme lui-même, qui permet de passer à l'échelle.

5.3 Approche décentralisée

L'expérimentation décentralisée met en lumière l'importance capitale de la formulation du vecteur d'état. L'absence initiale de l'orientation (*heading*) dans la représentation 10D créait des états aliénés (*aliased states*) : l'action de pivoter modifiait la direction du robot dans l'environnement sans altérer son vecteur PyTorch, piégeant l'agent dans des boucles de rotation infinies. L'extension à 11D a rétabli la propriété de Markov.

De plus, l'immobilité observée sur IDQN illustre un biais classique du *Reward Shaping* par distance Manhattan. Pour contourner une étagère, le robot doit temporairement s'éloigner de sa cible, ce qui lui inflige une pénalité de progression ($-0,32$). L'algorithme Q-learning converge alors vers un minimum local de moindre souffrance : rester immobile n'inflige qu'une pénalité minimale de pas ($-0,02$), évitant les risques de mauvaise direction.

5.4 Limites

Chaque configuration n'a été entraînée qu'avec une seule graine, faute de temps : les écarts entre PPO et le glouton (4 à 8 livraisons) dépassent largement le bruit d'évaluation (0,2 à 0,45 livraison), mais nous n'avons pas mesuré la variance d'un entraînement à l'autre. Le modèle évalué sur big date d'une version antérieure du réseau, sans relations entre paires de robots.

L'approche décentralisée a été entraînée sur un volume restreint d'épisodes (400 à 1200) en raison des contraintes de calcul CPU sur agent unique. Ce volume s'est avéré insuffisant pour permettre aux algorithmes à gradient de politique (IPPO/MAPPO) de dépasser la phase d'exploration stochastique dans les zones de forte congestion spatiale.

5.5 Perspectives

Une méthode comme QMIX [3], qui combine des valeurs par robot en une valeur jointe, se situe entre la table centralisée et des apprenants indépendants, et permettrait de mesurer ce qu'apporte la coordination explicite. Ajouter une action « attendre » aiderait les robots à se céder le passage, à condition de casser la symétrie entre deux robots identiques qui choisiraient d'attendre en même temps.

6 Conclusion

Nous avons étudié la coordination d'une flotte de robots dans un entrepôt selon deux approches. Côté centralisé, la Q-table jointe sature dès 3 robots, tandis que PPO, en raisonnant robot par robot plutôt que sur l'état joint, dépasse une heuristique de plus court chemin à toutes les tailles de flotte, avec un écart qui croît avec la taille de la flotte, et généralise à des labyrinthes jamais vus. Ce résultat tient autant à des choix simples, mesurés un par un, qu'à l'algorithme : masquer les actions impossibles, donner au réseau la distance à sa cible plutôt que la carte entière, et lui signaler ses voisins immédiats.

Côté décentralisé, l'utilisation de réseaux autonomes par robot supprime l'explosion combinatoire de l'espace d'actions. Nous avons démontré qu'une modélisation rigoureuse de l'état (intégration du *heading* et cible dynamique) combinée à un masque d'actions strict élimine totalement les actions invalides (0 sur IDQN). Cependant, la non-stationnarité et les puits de potentiel de la fonction de récompense piègent les agents indépendants dans des politiques passives ou des verrous spatiaux en l'absence de coordination explicite.

[À compléter ensemble : laquelle des deux approches recommander, et dans quelles conditions.]

Références

[1] R. Bellman. *Dynamic Programming.* Princeton University Press, 1957.

[2] S. Huang et S. Ontañón. A closer look at invalid action masking in policy gradient algorithms. In *Proceedings of the International FLAIRS Conference*, vol. 35, 2022.

[3] T. Rashid, M. Samvelyan, C. Schroeder de Witt, G. Farquhar, J. Foerster et S. Whiteson. QMIX : Monotonic value function factorisation for deep multi-agent reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, p. 4295–4304, 2018.

[4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford et O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv :1707.06347*, 2017.

[5] M. Tan. Multi-agent reinforcement learning : Independent vs. cooperative agents. In *Proceedings of the 10th International Conference on Machine Learning (ICML)*, p. 330–337, 1993.

[6] C. J. C. H. Watkins et P. Dayan. Q-learning. *Machine Learning*, 8(3–4) :279–292, 1992.

[7] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540) :529–533, 2015.

[8] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, et P. Wu. The surprising effectiveness of PPO in cooperative multi-agent games. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, p. 24611–24624, 2022.

[9] C. Schroeder de Witt, B. Peng, A. Kamienny, P. Torr, W. Böhmer et S. Whiteson. Is independent PPO all you need ? On the scaling of decentralized reinforcement learning. *arXiv preprint arXiv :2011.10829*, 2020.

[10] M. L. Littman. Markov games as a framework for multi-agent reinforcement learning. In *Machine Learning Proceedings 1994*, p. 157–163. Morgan Kaufmann, 1994.